

Internship Domain: Cyber Security

Focus Area: Static Application Security Testing (SAST) & Manual Code Review

Target Application: Python-Based Transit Ticketing API Prototype

1. Vulnerability Assessment Summary

A manual security code review was performed on the initial prototype of the booking software. Two critical security vulnerabilities were discovered during code inspection that could allow attackers to forge valid tickets or bypass authentication controls.

Vulnerability Log Table

Vulnerability ID	Severity	Flaw Category	Impact
SEC-01	Critical	Cryptographic Weakness (Broken Hashing)	Ticket forgery, financial loss, system-wide bypass.
SEC-02	Medium	Use of Deprecated/Unsafe Functions	Time-of-check to time-of-use (TOCTOU) synchronization issues.

2. In-Depth Vulnerability Analysis & Findings

Flaw 1: Broken Cryptographic Signature Generation (SEC-01)

- The Vulnerability:** The application uses standard hashlib.sha256() on a predictable string payload without a secret salt token or Keyed-Hash Message Authentication Code (HMAC).
- The Risk:** Because the payload pattern (passId|passengerId|price) is totally predictable, an advanced user can manipulate their priceUSD down to \$0.01, manually calculate the SHA-256 hash using an online tool, and inject it as a valid securitySignature. The scanner would accept it as authentic, resulting in catastrophic pricing fraud.

Flaw 2: Use of Deprecated/Insecure Time Routines (SEC-02)

- The Vulnerability:** Utilizing un-zoned datetime parameters (datetime.utcnow()) which triggers platform deprecation warnings.

- **The Risk:** Naive timezone handling causes synchronization vulnerabilities depending on server system clocks, allowing expired passes to pass validation gates across different regional boundaries.

3. Remediation & Secure Coding Implementation

To fix these flaws, the application code was refactored to implement defensive coding best practices:

1. **HMAC-SHA256 Integration:** Upgraded the hashing logic to use a protected server-side key (hmac). This guarantees signatures cannot be falsified by a client.
2. **Timezone-Aware Datetime:** Switched timestamp recording to strict UTC timezone-aware objects (`datetime.now(timezone.utc)`).

The screenshot shows a Visual Studio Code editor with a Python file named `secure_review.py` open. The code defines a function `create_secure_bus_pass` that takes `passenger_id` and `pass_type` as arguments. It generates a token payload, creates an HMAC signature using `hmac.new` with a system secret key, and returns a record containing the passenger ID, pass type, price, issue date, status, and security signature.

The terminal window shows the execution of the script. It prompts for a passenger ID (123) and a pass type (Single). It then displays the constructed metadata and the cryptographically signed output payload, which includes a JSON object with fields like `passId`, `passengerId`, `type`, `priceUSD`, `issueDate`, `status`, and `securitySignature`.

```

12 def create_secure_bus_pass(passenger_id, pass_type):
28     token_payload = f"({pass_record['passid']} {pass_record['passengerid']} {pass_record['priceUSD']})"
29
30     secure_hmac = hmac.new(SYSTEM_SECRET_KEY, token_payload.encode(), hashlib.sha256)
31     pass_record["securitySignature"] = secure_hmac.hexdigest()

```

```

PS C:\Users\jainm> python secure_review.py
=====
SECURE AUDIT APP VERIFICATION
=====
Enter Passenger ID: 123
Enter Pass Type (Single/Monthly): Single

[SECURE RUN] Constructing verified pass metadata for: 123
✓Transaction signature locked with authentic HMAC key verification.

--- Cryptographically Signed Output Payload ---
{
  "passId": "PASS-3F05A21D",
  "passengerId": "123",
  "type": "Single",
  "priceUSD": 15.0,
  "issueDate": "2026-06-06T10:00:16.307987+00:00",
  "status": "ACTIVE",
  "securitySignature": "2fae4663f8594fdded2782ab65be5d1d5ab97dfe2d430f7f8806042485f4a1abd"
}
PS C:\Users\jainm>

```